

企業向けエージェント型AIのエコノミクスとアーキテクチャの構造的限界：MicrosoftとUberの事例から読み解く技術的負債と次世代最適化戦略

1. 序論：定額制サブスクリプション経済から従量課金ユーティリティ経済への移行

2026年上半期、エンタープライズソフトウェア開発の領域において、これまで水面下で進行していた重大な構造的問題が表面化した。「AIによる生産性向上」というスローガンの裏で、自律的にコードを解析・生成するエージェント型AIの稼働コストが、企業の想定をはるかに超える規模で爆発的に増大したのである。

この事態を最も象徴する出来事が、世界最大のソフトウェア企業であるMicrosoftにおける内部決定である。2026年5月、同社はWindows、Microsoft 365、Outlook、Teams、Surfaceなどを管轄する主力部門「Experiences + Devices」のエンジニアに対し、AnthropicのAIコーディングツール「Claude Code」の利用を同社の会計年度末である6月30日までに停止するよう命じた¹。公式な説明は「ツールチェーンの統一」とされ、エンジニアは自社製の「GitHub Copilot CLI」への移行を義務付けられたが、真の理由は明白な財務的圧迫であった²。社内エンジニアが熱心にClaude Codeを使用した結果、Anthropicへのトークン課金支払いが想定を大きく上回り、Microsoftほどの資金力を持つ企業でさえ、そのユニットエコノミクス（単位経済性）を正当化できなくなったのである⁵。

ほぼ同時期に、配車サービス大手のUberでも同様の財務的危機が露呈した。同社の最高技術責任者（CTO）であるPraveen Neppalli Naga氏は、2026年通期のAIコーディングツール予算がわずか4ヶ月で完全に枯渇したことを認めた⁸。これらは単なる「ツールの使いすぎ」という表層的な問題ではない。エンタープライズAI市場が、従来の「1ユーザーあたりの定額制（Seat-based license）」から、コンピュータ資源の消費量に直結する「従量課金ユーティリティ経済」へと不可逆的な移行を遂げたことを示す歴史的な転換点である²。

本稿は、この事態の根本原因を技術構造の観点から分析する。エージェント型AIのトークン消費量は、対象となるコードベースのアーキテクチャとデータフォーマットに決定的に依存する。特に、長年にわたりエンタープライズシステムの標準とされてきたOfficeエコシステムとC#/I.NETアーキテクチャの組み合わせは、AI時代において構造的に最悪のコストを生み出す技術的負債となっている。本レポートでは、この構造的・経済的な不適合を解き明かし、AI時代の新たな技術選択と最適化戦略について論じる。

2. エージェント型AIの能力とトークン消費の爆発メカニズム

MicrosoftやUberが直面したコスト爆発の要因を理解するためには、まずエージェント型AIの振る舞いと、それがもたらすトークン消費のメカニズムを正確に把握する必要がある。

2.1 パターン適用から自律的推論への進化とCodexの変遷

これまでのAIコーディングツールは、基本的に「パターン認識と適用」に基づくコード補完（オートコンプリート）であった。2021年に登場した初期のOpenAI Codexは、GitHub Copilotの裏側で稼働し、開発者が入力したコードの続きを行単位で予測するツールとして普及した¹³。しかし、この初期型Codexモデルは2023年3月に非推奨となり、APIの提供も終了した¹³。

それに代わって2025年以降に台頭したのが、新しいCodexやClaude Codeに代表される「エージェント型AI」である¹⁴。これらはクラウド上のサンドボックス環境で並列タスクを実行し、単一の行を補完するのではなく、リポジトリ全体の構造を把握した上で多段階の変更計画を立案する¹⁴。エージェントは自律的にディレクトリ構成を読み解き、修正対象のファイルが依存する全クラス、設定ファイル、テストコードを芋づる式にコンテキストウィンドウに読み込む。システム全体のアーキテクチャを内部モデル化し、「このインターフェースを変更すれば、他のサービスにも修正が必要になる」と先回りして推論を行う。

この「深く読む」能力こそが、表面的な文法補完を超えた高いコーディング能力の源泉である。エンジニアがこれらのツールを熱狂的に支持し、Microsoft社内で「人気になりすぎた」と表現されるほど浸透したのは当然の結果であった²。しかし、AIにとって「読む」ことはコンテキストウィンドウへのトークン投入であり、トークン消費は直接的な課金に直結する。

2.2 Uberにおける予算枯渇とゲーミフィケーションの罠

Uberの事例は、エージェント型AIがエンタープライズ環境でいかに急速に普及し、そしていかに予測不可能なコストをもたらすかを示す完璧なケーススタディである。Uberは2025年12月に約5,000人のエンジニアを対象にClaude Codeを導入した¹⁰。

時期	UberにおけるClaude Codeの利用状況と財務的影響
2025年12月	エンジニアリングチームへのClaude Code導入開始
2026年2月	エンジニアの32%が利用を開始
2026年3月	利用率が84%に急増し、大多数が「エージェント的ユーザー」に分類される
2026年4月	利用率が95%に到達。コミットされたコードの70%がAI由来となる
2026年4月末	2026年通期のAIコーディングツール予算が完全に枯渇

この急速な普及の裏には、Uber社内で導入された「ゲーミフィケーション」の存在があった。同社はAIツールの利用量をチーム間で競わせる社内リーダーボードを展開し、トークン消費量が多いほどスコ

アが高くなる仕組みを設けた⁸。結果として、エンジニアにはトークン消費を最適化するインセンティブが働かず、むしろ無駄なトークンを浪費する動機付けが行われてしまった。

エージェントの並列処理能力もコストを跳ね上げた。ある修正タスクのために、Claude Codeは20から50、ときには100以上のファイルを読み込む。Naga CTO自身が実施したわずか2時間のデモセッションにおいて、1,200ドルものAPIコストが消費された事実は、このツールの財務的変動性の高さを雄弁に物語っている⁸。ヘビーユーザーのエンジニア1人あたりの月額トークン消費は500ドルから2,000ドルに達し、年間予算をわずか数ヶ月で吹き飛ばしたのである⁸。

2.3 単位経済性の崩壊とオープンソース・エージェントの暴走

このトークン消費の爆発は、大企業に限った話ではない。Anthropicは2026年4月、消費者向けのClaudeサブスクリプションにおいて、「OpenClaw」と呼ばれる人気のオープンソースエージェントフレームワークの実行を禁止した²。その理由は、月額200ドルのMaxプランを契約しているユーザーがOpenClawを自律稼働させた結果、単一のインスタンスが1日あたり1,000ドルから5,000ドルに相当するAPIコストを裏側で発生させていたためである²。

Nvidiaの応用ディープラーニング担当副社長であるBryan Catanzaro氏が指摘するように、計算資源のコストが、それを使用する従業員の給与を上回る逆転現象がすでに発生している²。MITの分析によれば、現在のトークン価格体系においては、AIによる自動化が人間の労働力よりも実際に低コストとなるのは、代替が想定されていた業務のわずか4分の1程度にとどまる²。Gartnerの予測では、2026年の世界のAI支出は2.5兆ドルに達する見込みだが、コスト高を理由に多くのPoCが調達段階で頓挫しており、インフラストラクチャプロジェクトのうちビジネスケースを完全に実現できるのはわずか28%である²。これは、エンタープライズAIが「幻滅の谷 (Trough of Disillusionment)」に突入したことを明確に示している。

3. C#/I.NETアーキテクチャがAIに強いる構造的負荷

Microsoftが「Claude Codeのコストが正当化できない」と判断した背景には、単純なツールの利用頻度だけでなく、同社が抱える巨大なコードベースの構造的な問題が存在する¹。Microsoftの製品群 (Windows、Office、Teams、Outlookなど) は数十年にわたって積層してきたシステムであり、C++やC#/I.NETで構築されている。ここでは、特に現代のエンタープライズシステムで広く採用されているC#/I.NETが、AIの効率をどのように悪化させるのかを分析する。

3.1 ファイル分散設計と抽象化レイヤーの読み込み連鎖

C#/I.NETの典型的な業務システムは、Clean Architecture、ドメイン駆動設計 (DDD)、SOLID原則といったソフトウェア工学のベストプラクティスに従って構築されている。しかし、これらの「人間にとっての正しい設計」は、AIにとってはコンテキストウィンドウを不必要に消費する「最悪の設計」となる。

1つの機能 (例えば顧客の受注登録) を実装するために、エンタープライズ構成ではエンティティ、データ転送オブジェクト (DTO)、リポジトリインターフェースとその実装、サービスインターフェースとその実装、APIコントローラ、バリデーションクラス、AutoMapperの設定、そしてDI (依存性の注入) コンテナへの登録など、10から15のファイルへの分散が標準的に行われる。

エージェント型AIがこの機能を修正する際、インターフェースを認識した後にDIコンテナの設定を参照して具象クラスを特定し、さらに依存する別のインターフェースへと芋づる式に展開していく必要がある。1つのクラスを正しく理解するために数十のファイルを読み込み、それぞれ数百行のコードをコンテキストとして保持しなければならない。この抽象化レイヤーの多重化が、構造的なトークン浪費

を引き起こす。

3.2 バリエーション型(判別共用体)の不在とクラス階層の肥大化

C#のエコシステムがAIに強い最も深刻な構造的負債は、長年にわたる「判別共用体(Discriminated Unions)」のネイティブサポートの不在である¹⁹。業務データには本質的に「複数のパターンのいずれか」という直和型の性質がある(例: 支払いは現金かカードか振込のいずれか)。F#、Rust、TypeScript、Swiftなどの言語では、これを1つの型定義で簡潔に表現できる。

しかし、C#ではこれらを表現するために、抽象基底クラスと複数の派生クラス、そしてそれら进行处理する巨大なswitch文(あるいはパターンマッチング)を用いたクラス階層を構築せざるを得ない¹⁹。この構造を修正するためには、基底クラス、すべての派生クラスファイル、ビジネスロジックを含むファイル、Entity Frameworkの継承戦略(TPH/TPTなど)のマッピング設定、JSONシリアライズのカスタムコンバーターなど、30から50に及ぶファイルをAIに読み込ませる必要がある。

Microsoftの言語設計チームもこの問題の深刻さを認識しており、2026年後半にリリース予定の.NET 11(C# 15)プレビューにおいて、長年要望されていたunionキーワードをついに導入するロードマップを示した¹⁹。この提案では、コンパイラによって網羅性が保証されるC#ネイティブの型ユニオンが導入される²⁴。しかし、これは新規開発には恩恵をもたらすものの、世界中の企業が抱える既存の莫大なC#コードベースの大部分は、依然として肥大化したクラス階層で構成されており、AIエージェントによる保守コストを跳ね上げる元凶として残り続ける。

3.3 関連ライブラリの設定地獄とC++のさらなる重圧

さらに、C#/.NETエコシステムは、プロジェクト設定(.csproj)、環境別設定(appsettings.json)、マイグレーションファイル、DI登録(Startup.cs/Program.cs)など、コードの動作に影響を与える設定ファイルが散在している。AIは正確な修正を行うためにこれらすべての全体像を把握しなければならない。

また、Microsoftの内部システムのように、UIコードまでC++で書かれている領域(MFC、ATL、Win32 APIなど)では状況はさらに悪化する。ヘッダーファイル(.h)と実装ファイル(.cpp)の分離、#includeの連鎖、マクロやプリプロセッサの存在により、AIエージェントの読み込み負荷はC#以上に増大する。これが、Microsoftほどの企業でもコストを正当化できなかった技術的背景である。

4. データフォーマットの非効率性: OOXMLとバイナリの罠

コードベースの構造に加えて、Microsoftエコシステムにおけるもう一つの巨大なトークン浪費要因が、Officeファイル形式(OOXML)の存在である。

4.1 OOXMLの過剰なXML処理オーバーヘッド

.xlsx、.docx、.pptxといったOfficeファイルは、表面上は構造化されたデータに見えるが、その実体はOffice Open XML(OOXML)という巨大なZIP圧縮XMLの集合体である。たった1つのセルに値が入力されているだけのExcelファイルであっても、その内部にはコンテンツ定義、スタイル定義、関係性メタデータなど、多数のXMLファイルが複雑なネームスペースと共に詰め込まれている。

AIエージェントにこのようなファイルを読み解かせる場合、意味のあるわずかなデータを取り出すために、XML構造の解析だけで膨大なトークンを消費する。これに対し、最新のデータサイエンスエコシステムで標準となっているParquet形式は、列ごとの型情報(スキーマ)をファイル内部に明示的に保持している。AIはParquetファイルの仕様を事前学習で完全に把握しており、追加のスキーマ説明

なしに、数分の一から数十分の一のトークン消費でデータを正確に理解できる。スキーマを持たず型の推論に依存するCSV形式と比較しても、ParquetやSQLite、JSONといった「スキーマを内包する標準フォーマット」はAI時代に圧倒的に有利な選択肢である。

4.2 オフィスエコシステムにおける横断的処理の累積コスト

実際の業務環境において、ファイルは単独で存在するわけではなく、SharePointやTeamsの共有フォルダに数千規模で蓄積されている。AIに対して「過去3年分の見積書から特定の取引履歴を抽出する」といったタスクを依頼した場合、AIはファイルの探索、ZIP解凍、XML構造解析、スタイル定義の読み飛ばし、テキスト抽出という極めてオーバーヘッドの高いプロセスをファイルごとに実行しなければならない。

さらに、多くの日本企業のファイルサーバーに残存している旧形式のバイナリファイル(.doc、.xls)を扱う場合、AIは専用のパーサーを経由してデータを変換する必要がある、この変換層自体がさらなるトークンを消費する。

4.3 OfficeとC#/(.NETの相乗的コスト増幅

エンタープライズ基幹システムにおいては、C#/(.NETのコードとOfficeファイルが密接に結合している。受注データのExcelインポートや、月次レポートのPowerPoint生成など、業務ロジックはOfficeファイルの入出力を伴う。

このようなシステムをAIが保守・拡張する場合、AIは(1)C#の高度に分散された抽象化レイヤー、(2)OOXMLの複雑なデータ構造、(3)両者をつなぐマッピングロジック(ExcelInteropやOpenXML SDKなど)の三層を横断的に読み込む必要がある。これらが組み合わさることで、トークン消費は線形的な加算ではなく、指数関数的な相乗効果をもって爆発する。これが、「正しく設計された」既存の基幹システムがAI時代に直面する構造的危機の本質である。

5. 価格モデルの転換と技術的緩和策の限界

5.1 GitHub AI Creditsへの移行とコストの転嫁

こうしたアーキテクチャの不適合が生み出す推論コストの増大に耐えかね、プラットフォーム側もビジネスモデルの変更に踏み切った。GitHubは2026年6月1日より、Copilotの料金体系を従来の無制限な「プレミアムリクエスト」モデルから、使用量に基づく「GitHub AI Credits」モデルへと移行する²⁶

。この新体系では、Copilot Business(月額19ドル)やCopilot Enterprise(月額39ドル)には同額分のクレジット(1クレジット=0.01ドル)が付与され、組織内でプールされる²⁶。しかし、これを超過した分については、入力トークン、出力トークン、そしてキャッシュされたトークンのすべてに対して厳密な従量課金が行われる²⁶。

モデル	カテゴリ	入力 (100万 トークン)	キャッシュ入力 (100万トーク ン)	出力 (100万 トークン)
GPT-4.1	Versatile	\$2.00	\$0.50	\$8.00

GPT-5.4	Versatile	\$2.50	\$0.25	\$15.00
GPT-5.4 mini	Lightweight	\$0.75	\$0.075	\$4.50
GPT-5.5	Powerful	\$5.00	\$0.50	\$30.00

AnthropicのClaude Opus 4.7モデルなどが採用している新しいトークナイザーは、同じプロンプトに対しても最大35%多くのトークンを消費する設計となっており、プラットフォームの卸売コストの上昇がそのままユーザー企業に転嫁される構造となっている²⁹。インドのある開発チームの検証では、新モデル移行後の請求額が従来の約9倍に達すると予測されており、予算管理の抜本的な見直しが必要と迫られている²⁹。

5.2 プロンプトキャッシング(KVキャッシュ)技術とその経済効果

このトークン浪費問題に対する主要な技術的緩和策として、AIプロバイダー各社は「プロンプトキャッシング(Prefix Caching)」技術を導入している³¹。これは、システムプロンプトや変化しない巨大な背景情報(ファイルのコンテキストなど)の内部計算結果(KVテンソル)を保存し、後続のリクエストで再計算をスキップする技術である³²。

プロバイダー	キャッシュ技術の特徴とコスト削減効果
Anthropic	キャッシュ読み取りコストは100万トークンあたり0.30（通常の3.00から90%削減）。ブロックは1,024トークン以上で指定。デフォルト保持は5分間 ³¹ 。
DeepSeek	V2のMLAアーキテクチャを活用した「Context Caching on Disk」をデフォルト有効化。キャッシュヒット時のコストは\$0.014/100万トークンと極めて安価 ³⁶ 。
OpenAI/Google	OpenAIは自動適用で50%割引、Google Vertex AIも暗黙的・明示的なキャッシングをサポート ³² 。

あるセキュリティ監査プロジェクトでは、プロンプトキャッシングの導入によりキャッシュヒット率を7%から84%へと劇的に向上させ、全体のコストを70%削減することに成功した³³。また、TTFT(Time to First Token)が50~85%短縮され、レスポンスの遅延も大幅に改善される³²。

5.3 キャッシュ技術の限界とアーキテクチャへの依存性

しかし、プロンプトキャッシングは万能の解決策ではない。キャッシュが有効に機能するためには、静的なコンテンツ(システムプロンプトや設定)をプロンプトの先頭(上部)に配置し、動的なコンテンツ(会話履歴や変更対象コード)を末尾(下部)に配置するという厳密な順序付けが必要である³¹。DeepSeekの仕様が示すように、後続のリクエストはキャッシュされたプレフィックス単位と完全に一致した場合にのみヒットする(スライディングウィンドウアテンションの制約)³⁶。C#/I.NETのような高度にファイルが分散されたシステムでは、タスクごとに読み込むファイルの組み合わせが動的に変化するため、プレフィックスの一貫性を保つことが極めて困難である。結果としてキャッシュミスが頻発し、毎回ゼロからトークンを消費する事態に陥る。つまり、キャッシュ技術の恩恵を最大限に受けるためにも、アーキテクチャ自体が「静的でまとまった構造」を持っている必要があるのだ。

6. 認知パラダイムの転換: パターン認識から構造推論へ

エージェント型AIの普及は、ソフトウェアエンジニアの働き方そのものに根本的な認知の転換を要求している。

6.1 Codexが残した教訓とエッジケースへのバグの押し込み

従来のプログラミングは、本質的に「既存のパターンの暗記と適用」であった。初期のCodexのようなパターン適用型のAIは、この作業を完璧に代行した。しかし、Codexを中心とした開発には致命的な副作用があった。AIは「とりあえず動くコード」を生成することを優先するため、矛盾する要件や曖昧な仕様に直面した際、不整合を「テストしにくい場所」に押し込む傾向がある。

エラーハンドリングの漏れ、非同期処理の競合状態、システム境界での異常系処理など、エッジケースに隠蔽されたバグは、本番環境の高負荷状態でのみ発症し、その原因追跡は極めて困難となる。Codexが生成した「もっともらしいが間違っている」コードのデバッグは、パターン認識ではなく、システム全体の構造的因果関係を推論する能力を必要とする。

6.2 構造を考える時代と「ビルダー」への進化

現在台頭しているClaude Code等の強みは、まさにこの「構造的推論」と「設計判断」にある。AI時代における人間の役割は、個々のツール(IDE、ORM、Officeアプリ)の細かい使い方を暗記することから、システム全体の「構造を設計すること」へとシフトしている。

どのようなデータ形式を採用するか、どこまでを標準フォーマットで直叩きし、どこに抽象化を設けるか。これをプレーンなテキスト(Markdown)で仕様化し、AIに明確なコンテキストを与える能力が求められる。「リベラルアーツ(構造を読む力)」へのシフトである。

しかし、長年「正しい設計(SOLID、Clean Architectureなど)」を内面化してきたベテランエンジニアほど、パターンの外に出る訓練を受けておらず、この転換に苦戦する。過剰な抽象化を「美しい」と感じる直感そのものが、AI時代には高いトークンコストをもたらす足かせとなる。

7. 次世代最適化戦略: Pythonと標準フォーマットへの経済的収斂

Office + C#/I.NETの組み合わせが構造的な高コストを生む一方で、その対極に位置するアーキテクチャが圧倒的な経済的優位性を確立しつつある。それは「Markdown + Python + 標準フォーマット(JSON、SQLite、Parquet)」の組み合わせである。

7.1 Pythonが持つ5つの構造的優位性

AI時代において、Pythonは単なる好みの言語ではなく、経済的必然として選ばれる。その理由は以下の5点に集約される。

1. 学習データの圧倒的優位: AI基盤モデルの訓練データにおいてPythonは最大派閥であり、生成精度が他言語に比べて極めて高い。少ない試行回数で目的のコードを出力するため、プロンプトの往復によるトークン消費が最小化される。
2. ボイラープレートの排除: 型宣言、波括弧、複雑な修飾子が不要であり、同一のロジックを表現するために必要な文字数(トークン数)が構造的に少ない。
3. データ構造との親和性: PythonのdictはJSONと透過的に変換可能であり、Pandas/PolarsのDataFrameはParquetやSQLiteとネイティブに連携する。中間に巨大なマッピング層(ORMやDTO)を挟む必要がない。
4. 1ファイル完結型の設計: 複雑なクラス階層を作らずとも、関数ベースのフラットな設計でドメインロジックを完結させやすい。AIが読み込むべきコンテキストが単一ファイルに収まる。
5. AIEコシステムの中心: LangChain、Hugging Face、各種Agent SDKなど、AI自身が「Pythonでの実装方法」を最も深く熟知している。

7.2 個人ビルダーと小規模組織の非対称的優位性

大企業が数千ドルのトークン課金に苦しむ中、個人ビルダーや小規模開発者はこの嵐を恐れる必要がない。Pythonと標準フォーマットで構築されたシステムは、AIが読み込むファイル数が極端に少なく、プロンプトキャッシングのヒット率も高水準で安定する。

フロントエンドにおいてDart/Flutterのような型システムと深いツリー構造を持つ言語を使用する場合、トークン消費はやや増加する傾向にあるが、それでもバックエンドをPythonでフラットに保つことで、全体のコストは相殺される。大企業がレガシーなアーキテクチャの維持に莫大なAI予算を浪費している一方で、小規模開発者は月額100ドルから200ドル程度の定額プランの範囲内で、同じClaude Code等のエージェント型AIをフル活用し続けることが可能である。この非対称性こそが、AI時代における小規模組織の最大の構造的優位である。

8. 結論: 技術選択は経営財務的決断である

AI時代の到来は、エンタープライズにおける技術選択の意味を根本から変容させた。Microsoftが自社の基幹部門でClaude Codeを停止し、Uberが予算を早期に枯渇させた事象は、次のような冷徹な事実を証明している。

1. アーキテクチャの不適合は財務的危機に直結する: Office + C#/ .NETという、これまで「正しい設計」とされてきた抽象度の高い分散型アーキテクチャは、エージェント型AIのトークン消費メカニズムと決定的に相性が悪く、構造的に高コストを生み出す。
2. コストはモデルの進化では解決しない: AIが高機能になり、自律的な推論能力を高めるほど、読み込むべきコンテキストの量は増加する。プロンプトキャッシングなどの緩和策は存在するが、分散化されたアーキテクチャのもとではその効果は限定的である。
3. 従量課金リスクの企業側への転嫁: GitHub CopilotのAIクレジット制移行に見られるように、非効率なコードベースをAIに読ませるコストは、プラットフォームからユーザー企業のバランスシートへと直接転嫁されるようになった。

企業にとっての選択肢は既に明確に提示されている。派手な新技術を追い求めることなく、Markdown、Python、そしてスキーマを内包する標準フォーマット(Parquet、SQLite、JSON)の直叩きという「古くからある標準への回帰」である。このシンプルでフラットなアーキテクチャこそが、AIEコシステムのコンテキスト消費を最小化し、ユニットエコノミクスを成立させる唯一の合理的な戦略であ

る。

Microsoftほどの巨大企業ですら、自社エコシステムが積層しすぎた結果、AIによる保守コストを正当化できなくなった。同じ構造的負債を抱える多くの企業が、近い将来、同じ財務的な嵐に見舞われるのは確実である。AI時代における「品質」とは、複雑なデザインパターンを駆使することではなく、AIが読みやすく、キャッシングが機能しやすく、不整合をテストしにくい場所に隠さない「シンプルで透過的な構造」を設計できるかどうかにかかっている。技術の選択は、もはやエンジニアリングの好みの問題ではなく、企業がAI時代を生き残るための最も重要な経営判断である。

引用文献

1. AI bubble warning signs? Microsoft, Uber face unexpected coding agent cost surge in 2026, 5月 26, 2026にアクセス、
<https://www.thenews.com.pk/latest/1403757-ai-bubble-warning-signs-microsoft-uber-face-unexpected-coding-agent-cost-surge-in-2026>
2. Microsoft's quiet Claude Code retreat and the real cost of enterprise AI - TNW, 5月 26, 2026にアクセス、
<https://thenextweb.com/news/microsoft-claude-code-retreat-ai-cost>
3. Microsoft has cancelled internal Claude Code licenses after token billing became too expensive : r/GenAI4all - Reddit, 5月 26, 2026にアクセス、
https://www.reddit.com/r/GenAI4all/comments/1tln3jq/microsoft_has_cancelled_internal_claude_code/
4. Microsoft cancels Claude Code licenses, shifting developers to GitHub Copilot CLI — a move likely driven by financial motives | Windows Central, 5月 26, 2026にアクセス、
<https://www.windowscentral.com/microsoft/microsoft-cancels-claude-code-licenses-shifting-developers-to-github-copilot-cli-a-move-likely-driven-by-financial-motives>
5. Microsoft Drops Claude Code After Budget Overrun | AI Weekly, 5月 26, 2026にアクセス、
<https://aiweekly.co/alerts/microsoft-drops-claude-code-after-budget-overrun>
6. Microsoft canceled its internal Claude Code licenses this week after token-based billing made the cost untenable - Reddit, 5月 26, 2026にアクセス、
https://www.reddit.com/r/theprimeagen/comments/1tjztiw/microsoft_canceled_its_internal_claude_code/
7. Microsoft CEO sends shocking message to IT employees - TheStreet, 5月 26, 2026にアクセス、
<https://www.thestreet.com/technology/microsoft-ceo-sends-shocking-message-to-employees>
8. Claude Code Drained Uber's AI Budget in Months - DesignRush News, 5月 26, 2026にアクセス、
<https://news.designrush.com/uber-2026-ai-budget-claude-code-token-spend>
9. AI promised cost savings, but Microsoft and Uber say it's costing more than human workers, 5月 26, 2026にアクセス、
<https://www.livemint.com/companies/news/ai-was-supposed-to-cut-costs-microsoft-and-uber-are-finding-it-is-more-expensive-than-paying-human-employees>

[-11779666290918.html](#)

10. 5月 26, 2026にアクセス、
https://www.reddit.com/r/artificial/comments/1t1mxx6/uber_burned_its_entire_2026_ai_coding_budget_in_4/#:~:text=jimmytoan-,Uber%20burned%20its%20entire%202026%20AI%20coding%20budget%20in%204.2k%20per%20engineer%20per%20month&text=Uber%20deployed%20Claude%20Code%20to,off%20faster%20than%20anyone%20planned.
11. Uber Torch's Entire 2026 AI Budget on Claude Code in Four Months - Briefs Finance, 5月 26, 2026にアクセス、
<https://www.briefs.co/news/uber-torchs-entire-2026-ai-budget-on-claude-code-in-four-months/>
12. Microsoft cancels Claude Code licenses as AI costs surge across ..., 5月 26, 2026にアクセス、
<https://cryptobriefing.com/microsoft-cancels-claude-code-ai-costs/>
13. Is Codex available through the OpenAI API? - Milvus, 5月 26, 2026にアクセス、
<https://milvus.io/ai-quick-reference/is-codex-available-through-the-openai-api>
14. The Return of Codex AI — as an Agent - Visual Studio Magazine, 5月 26, 2026にアクセス、
<https://visualstudiomagazine.com/articles/2025/05/16/the-return-of-codex-ai-as-an-agent.aspx>
15. OpenAI Codex: From 2021 Code Model to a 2025 Autonomous Coding Agent - Medium, 5月 26, 2026にアクセス、
<https://medium.com/@aliazimidarmian/openai-codex-from-2021-code-model-to-a-2025-autonomous-coding-agent-85ef0c48730a>
16. What Is the OpenAI Codex Subscription Change? Why Third-Party Tools Lost Claude Access | MindStudio, 5月 26, 2026にアクセス、
<https://www.mindstudio.ai/blog/openai-codex-anthropic-subscription-change>
17. Uber burned its entire 2026 AI coding budget in 4 months - \$500-2k per engineer per month, 5月 26, 2026にアクセス、
https://www.reddit.com/r/artificial/comments/1t1mxx6/uber_burned_its_entire_2026_ai_coding_budget_in_4/
18. "Microsoft Azure AI" Related News - BigGo Finance, 5月 26, 2026にアクセス、
<https://finance.biggo.com/s/Microsoft%20Azure%20AI>
19. Discriminated Unions in C# 14: Modeling Data the Smarter Way | by Yaathavarajan Krishnarajah | Medium, 5月 26, 2026にアクセス、
<https://medium.com/@yathavarajan/discriminated-unions-in-c-14-modeling-data-the-smarter-way-6390e3d411c6>
20. C# Discriminated Unions and .NET Channels - Charles Chen (@chrlnchn), 5月 26, 2026にアクセス、
<https://chrlnchn.dev/blog/2024/07/csharp-discriminated-unions-and-dotnet-channels/>
21. C# 14 & Discriminated Union : r/csharp - Reddit, 5月 26, 2026にアクセス、
https://www.reddit.com/r/csharp/comments/1nawh26/c_14_discriminated_union/
22. C# 15 and the Future of .NET: A Comprehensive Deep Dive into Union Types and the Evolution of C# 14 | by Syed Qutub Uddin Khan - Medium, 5月 26, 2026にアクセス、

- <https://medium.com/@s.quddin/c-15-and-the-future-of-net-a-comprehensive-deep-dive-into-union-types-and-the-evolution-of-c-14-cd842f306233>
23. Discriminated Unions on ASP.NET 11 roadmap : r/dotnet - Reddit, 5月 26, 2026にアクセス、
https://www.reddit.com/r/dotnet/comments/1r0b5oi/discriminated_unions_on_aspnet_11_roadmap/
 24. Explore union types in C# 15 - .NET Blog, 5月 26, 2026にアクセス、
<https://devblogs.microsoft.com/dotnet/csharp-15-union-types/>
 25. Unions - C# feature specifications (preview) - Microsoft Learn, 5月 26, 2026にアクセス、
<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/proposals/unions>
 26. Usage-based billing for organizations and enterprises - GitHub Docs, 5月 26, 2026にアクセス、
<https://docs.github.com/copilot/concepts/billing/usage-based-billing-for-organizations-and-enterprises>
 27. As expected, GitHub Copilot is moving to usage-based billing : r/GithubCopilot - Reddit, 5月 26, 2026にアクセス、
https://www.reddit.com/r/GithubCopilot/comments/1sx8a8v/as_expected_github_copilot_is_moving_to/
 28. GitHub Copilot is moving to usage-based billing, 5月 26, 2026にアクセス、
<https://github.blog/news-insights/company-news/github-copilot-is-moving-to-usage-based-billing/>
 29. GitHub Copilot's new pricing could raise costs 9x from June 1: Best AI coding alternatives for developers, 5月 26, 2026にアクセス、
<https://www.hindustantimes.com/technology/github-copilot-ai-credits-billing-june-2026-best-coding-models-india-101779176274373.html>
 30. Models and pricing for GitHub Copilot, 5月 26, 2026にアクセス、
<https://docs.github.com/copilot/reference/copilot-billing/models-and-pricing>
 31. Prompt caching - Claude API Docs, 5月 26, 2026にアクセス、
<https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
 32. Prompt Caching Infrastructure: Reducing LLM Costs and Latency - Introl, 5月 26, 2026にアクセス、
<https://introl.com/blog/prompt-caching-infrastructure-llm-cost-latency-reduction-guide-2025>
 33. How We Cut LLM Costs by 59% With Prompt Caching - ProjectDiscovery.io, 5月 26, 2026にアクセス、
<https://projectdiscovery.io/blog/how-we-cut-llm-cost-with-prompt-caching>
 34. Prompt Caching is a Must! How I Went From Spending \$720 to \$72 Monthly on API Costs - Du'An Lightfoot, 5月 26, 2026にアクセス、
<https://labeveryday.medium.com/prompt-caching-is-a-must-how-i-went-from-spending-720-to-72-monthly-on-api-costs-3086f3635d63>
 35. Prompt Caching Explained: What It Is, What It Isn't, and When to Use It - Medium, 5月 26, 2026にアクセス、
<https://medium.com/@michael.hannecke/prompt-caching-explained-what-it-is->

[what-it-isnt-and-when-to-use-it-9f5c6fce7bdb](#)

36. Context Caching - DeepSeek API Docs, 5月 26, 2026にアクセス、
https://api-docs.deepseek.com/guides/kv_cache
37. DeepSeek API introduces Context Caching on Disk, cutting prices by an order of magnitude, 5月 26, 2026にアクセス、
<https://api-docs.deepseek.com/news/news0802>
38. Anyone here successfully lowering costs with "prompt caching" and/or "batch processing"? : r/Anthropic - Reddit, 5月 26, 2026にアクセス、
https://www.reddit.com/r/Anthropic/comments/1qzmjcx/anyone_here_successfully_lowering_costs_with/
39. Here's to hoping caching improves for generative writing and becomes as efficient as it is for coding : r/DeepSeek - Reddit, 5月 26, 2026にアクセス、
https://www.reddit.com/r/DeepSeek/comments/1tmpoxk/heres_to_hoping_caching_improves_for_generative/